Network Intrusion Detection with Machine Learning: A Comprehensive Pipeline using the KDD99 Dataset

Federico Calzolari

September 7, 2025

Contents

Chapter 1

Introduction

Network security has become a critical problem in the digital era, with cyber-attacks growing both in frequency and sophistication. Machine learning approaches capable of learning complex patterns directly from network data have gained significant attention for their potential to automate and optimize network attack detection.

This project is focused on the design, optimization, and evaluation of a machine learning pipeline for network intrusion detection, using the well established KDD99 dataset as a benchmark. The primary objective is to develop a robust workflow that can accurately classify network traffic and identify attacks, leveraging ML techniques and best practices to maximize real-world applicability and reliability.

A central theme of this work is the progression from a binary classification task, so that we can classify between normal and potentially malicious network connections, to a multiclass classification scenario, in which the goal is to precisely identify the specific type of attack.

Special carefulness is placed on the optimization process: preprocessing, feature selection, handling class imbalance, and hyperparameter tuning are integrated into the ML pipeline. Throughout the project, particular attention is paid to avoiding common pitfalls such as data leakage, and to ensuring that all reported metrics reflect realistic operational performance.

Chapter 2

Dataset and Problem Framing

2.1 The KDD99 Dataset

The KDD99 dataset is one of the most widely used benchmarks for research in network intrusion detection. Developed as part of the 1999 Knowledge Discovery and Data Mining (KDD) Cup competition, it simulates a real-world environment by providing a large collection of connection records generated from a variety of simulated attacks and normal network traffic.

Each record in the dataset corresponds to a single TCP/IP connection and is described by 41 features, which groups a range of characteristics:

- **Basic features** of individual TCP connections (e.g., duration, protocol type, service, source and destination bytes).
- **Content features** extracted from the data payload (e.g., number of failed logins, root access attempts).
- **Traffic features** computed using a two second time window (e.g., number of connections to the same host, percentage of connections with errors).

The target variable (label) indicates whether the connection is `normal` or belongs to one of several attack categories. These attacks are further grouped into four main types:

- **Denial of Service (DoS):** e.g., smurf, neptune, teardrop
- **Probe:** e.g., satan, ipsweep, nmap
- **User to Root (U2R):** e.g., buffer_overflow, rootkit
- **Remote to Local (R2L):** e.g., guess_passwd, warezclient

A key challenge with the KDD99 dataset is its high degree of **class imbalance**: the majority of records correspond to just a few types of attacks or normal connections, while many attack types are extremely rare. Additionally, the dataset

contains both continuous and categorical variables, as well as significant redundancy and some duplicate records, which necessitate careful preprocessing.

Despite its age and some known limitations, KDD99 remains a valuable resource for benchmarking new methods and developing best practices in network intrusion detection since many new attacks are derived from some fundamentals one, serving as a foundation for many comparative studies and competitions.

Label	**Count**	**Label**	**Count**	**Label**	**Count**	**Label**	**Count**
normal	87,832	teardrop	918	nmap	158	rootkit	10
neptune	51,820	satan	906	guess_passwd	53	loadmodule	9
back	968	warezclient	893	buffer_overflow	30	ftp_write	8
ipsweep	651	smurf	641	warezmaster	20	multihop	7
portsweep	416	pod	206	land	19	phf	4
imap	12	perl	3	spy	2		

Table 2.1: Label distribution in the dataset

2.2 Binary and Multiclass Classification

In the context of network intrusion detection, two principal classification tasks are considered: checking if an attack is being pursued and what type it is.

Binary Classification

The binary classification task distinguish between **normal network traffic** and **any form of attack**. Here, all attack types regardless of their specific characteristics are grouped together into a single "attack" category:

- **Normal:** Legitimate network connections.
- **Attack:** Any connection associated with malicious activity, regardless of its specific subtype.

This approach is particularly useful for rapid detection and alerting, where the priority is to flag suspicious activity as quickly as possible, potentially triggering further investigation or automated response.

Multiclass Classification

The multiclass classification task requires the model not only to detect attacks, but also to **identify the specific type of attack** among all the categories present in the dataset. In this scenario, the classifier must assign each connection to one of many possible classes (e.g., `normal`, `neptune`, `smurf`, `satan`, etc.).

In summary, the binary classification task prioritizes generic detection capability, while multiclass classification aims for comprehensive identification and characterization of network intrusions.

Chapter 3

Preprocessing Pipeline

3.1 Feature Engineering and Encoding

Building an effective classification pipeline for the KDD99 dataset requires a careful feature engineering. The raw dataset contains a mix of categorical, boolean, and continuous variables, many of which are highly skewed. To ensure robust learning and avoid bias, several preprocessing steps are applied:

- **Categorical Features:** Variables such as `protocol_type`, `service`, and `flag`, although nominal by nature, are encoded using *Ordinal Encoding* instead of One-Hot Encoding. This choice is not made to assume any ordinal relationship among the categories, but rather a practical necessity for compatibility with the SMOTENC algorithm, which requires categorical features to be represented as integer labels in order to distinguish them from continuous variables during oversampling. While One-Hot Encoding would be semantically ideal, it is not directly supported by SMOTE-based algorithms and may lead to invalid synthetic samples. To mitigate the potential bias introduced by the artificial ordering, the encoded features are only used within pipelines that handle categorical variables appropriately.

- **Log-Transformed Features:** Features with strong positive skew (such as `duration`, `src_bytes`, and `dst_bytes`) are subjected to a log transformation, which compresses large outliers and makes their distribution better for modeling.

- **Continuous Features:** All remaining continuous features are normalized into the [0,1] interval using MinMax scaling, which supports model convergence and prevents dominance by features with large numeric ranges.

A visual summary of the preprocessing workflow is provided below:

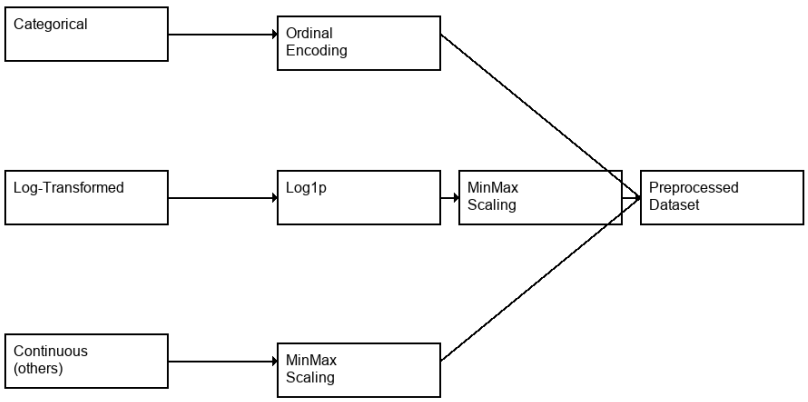


Figure 3.1: Minimal KDD99 Preprocessing Pipeline

3.2 Reproducibility and Data Leakage Avoidance

To ensure a correct evaluation and generalization, all transformations are encapsulated in a machine learning pipeline. When using tools such as `imbalanced-learn`'s `Pipeline` and `GridSearchCV`, each step (e.g., preprocessing, feature selection, classification) is automatically fit only on the training partition of each cross-validation split. The corresponding transformations are then applied to the test fold using parameters learned only from the training data, preserving the independence of the evaluation set.

This pipeline-based approach not only prevents leakage, but also guarantees the **reproducibility** of the experimental workflow: every transformation, hyperparameter, and random seed is recorded and can be systematically reapplied, making results comparable across different runs, parameter settings, and even datasets.

Chapter 4

Feature Selection

4.1 Model-Specific Feature Selection Strategies

In this project, feature selection was based with the type of predictive model to be used in the classification step.

- **Random Forest:** For the random forest classifier, feature selection is based on the model's feature importance scores. Multiple random forest models are trained with different random seeds, and their feature importances are averaged to obtain a ranking. The top features, according to this importance are selected for use in the final RF model.
- **Support Vector Machine (SVM):** Linear SVMs benefit most from a subset of features that maximize class separation in a linear space. For SVMs, Recursive Feature Elimination (RFE) is applied: a linear SVM is trained iteratively, each time removing the least important feature (according to model coefficients) until the optimal number of features is reached.
- **Logistic Regression:** Similar to the SVM approach, RFE is employed with logistic regression as the estimator.

4.2 Handling Imbalanced Data

A major challenge in the KDD99 dataset is the severe **class imbalance** present among labels. Class imbalance can significantly distort both model training and evaluation, we need a method to fix this issue.

Why SMOTENC? To mitigate these issues, a **resampling techniques** is applied. In this project, **SMOTENC** (Synthetic Minority Over-sampling Technique for Nominal and Continuous features) was used, a variant of SMOTE designed specifically for datasets containing both numerical and categorical variables.

Unlike the standard SMOTE implementation, which assumes that all input features are continuous, SMOTENC allows the user to specify which features are categorical. It then treats these variables appropriately (e.g., interpolating between `TCP` and `UDP`).

Categorical features such as `protocol_type`, `service`, and `flag` are first encoded into integer labels using *Ordinal Encoding*. While One-Hot Encoding is typically preferred for nominal attributes, it is incompatible with SMOTENC. The use of Ordinal Encoding in this context is a practical necessity: it enables oversampling while ensuring that SMOTENC recognizes the categorical nature of these features and treats them as discrete labels during synthetic sample generation. The encoded values are not used to imply any semantic ordering.

Resampling Protocol: To avoid data leakage, SMOTENC is applied in the pipeline and *inside* each fold of the cross-validation process, so that it is only applied only on training data after the split.

Class Pruning: Before training, all classes with fewer than *ten* total samples were removed from the dataset. This decision was made to ensure the reliability of resampling and evaluation metrics. Extremely low-support classes lead to unstable synthetic generation and disproportionately affect macro-averaged metrics such as F1-macro, introducing high variance and unrepresentative results, and ultimately make the whole classification process worse.

Chapter 5

Model Selection and Hyperparameter Optimization

5.1 Model Selection and Performance Comparison

Selecting the most suitable classification model is a critical step in designing an effective intrusion detection pipeline. Given the class imbalance and the multiclass nature of the task, F1 macro was chosen as the primary evaluation metric, as it equally weights the performance on all classes, regardless of their frequency.

Three different model were chosen:

- **Random Forest**
- **Logistic Regression**
- **Support Vector Machine (Linear SVM)**

For each model, the pipeline included standardized preprocessing and a dedicated, model-specific feature selection step. To ensure robust and realistic performance estimates, stratified 5-fold cross-validation was employed, providing a balanced assessment across all classes.

Results:

- **Random Forest** consistently achieved the highest F1 macro, indicating superior ability to detect and differentiate among all attack types as well as normal traffic.
- **SVM** and **Logistic Regression** had lower F1 macro scores and they will not be used in the following analysis.
- Standard deviations (error bars) were reported to account for variability across different folds.

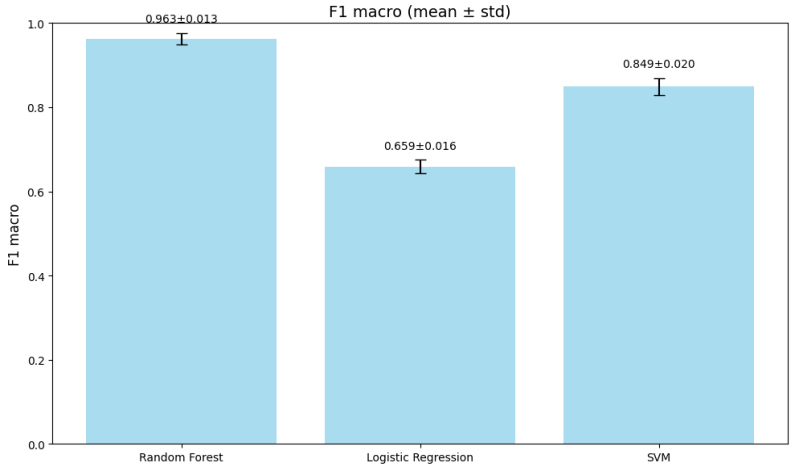


Figure 5.1: F1 macro mean and standard deviation for each model, 5-fold stratified cross-validation.

Based on these results, **Random Forest** was selected as the best-performing model for subsequent optimization and in-depth analysis. The high F1 macro score confirms its robustness in handling both class imbalance and complex feature interactions, making it the best model for the network intrusion detection scenario considered in this project.

5.2 Grid Search and Resource Constraints

In this project, a grid search approach was employed to optimize the configuration of the Random Forest classifier.

Grid Search Approach: The parameter space was defined by selecting only the most impactful hyperparameters for tuning:

- `n_estimators`: Number of trees in the forest ({100, 500})
- `max_depth`: Maximum depth of each tree ({20, None})
- `min_samples_leaf`: Balancing for class imbalance ({1, 5})

This resulted in a total of $2 \times 2 \times 2 = 8$ parameter combinations.

Resource Constraints and Subsampling: Because of the high dimensionality of the data, the size of the KDD99 dataset, and the limitations of available hardware, the search of parameters was necessarily reduced.

To manage computational cost, the grid search was limited to a manageable number of parameter sets, and only the most relevant parameters were included. Sample based tuning was considered as a compromise to allow for rapid experimentation on a subset of the data before applying the best found configuration to the full dataset.

Even if this approach may not find the optimal values, it was necessary to adopt this constraint for practical reasons.

Experimental Setup and Timing: Grid search was performed with stratified 3-fold cross-validation, ensuring balanced evaluation across all classes for each hyperparameter combination. With 8 combinations and 3 folds, a total of $8 \times 3 = 24$ fits were performed.

- **Number of parameter combinations:** 8
- **Number of folds:** 3
- **Total fits:** 24
- **Total duration:** Approximately 2 hour (on a laptop)

These constraints, were necessary to avoid excessive computation times and system instability. Despite the limited grid, meaningful improvements in model performance were achieved as shown in the following parts.

Chapter 6

Binary Classification Results

6.1 Pipeline Structure:

```
pipeline_binary = Pipeline([
  ('preprocessing', KDD99Preprocessor()),
  ('smote', SMOTENC(categorical_features=categorical_cols, random_state=42)),
  ('feature_selection', FeatureSelectorByModel(model_type='rf', threshold=0.3)),
  ('rf', RandomForestClassifier(**rf_best_params, random_state=42))
])
```

Listing 6.1: Binary pipeline code

1. **Preprocessing:** All categorical, boolean, and continuous features are processed by the custom `KDD99Preprocessor`, which applies ordinal encoding, log transformation of skewed features, and min-max scaling as appropriate.
2. **Resampling:** The SMOTENC technique is integrated for oversampling minority classes.
3. **Feature Selection:** The `FeatureSelectorByModel` transformer, configured for random forest feature importance, selects the most relevant subset of features based on cross-validated importance ranking.
4. **Classification:** A Random Forest classifier is trained on the transformed and balanced dataset.

Important note To study the binary classification problem, the label for attacks has been transformed into "attack," while the "normal" class label remains unchanged.

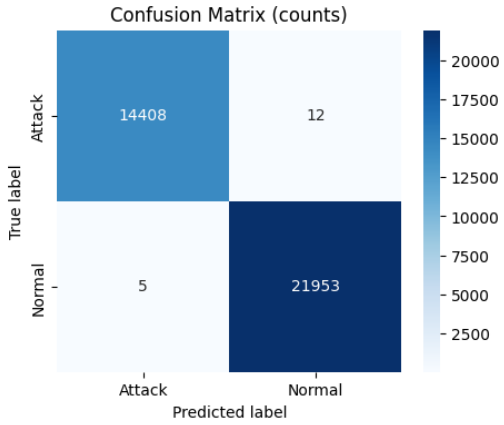
Best Parameter Choices: The grid search process identified the following hyperparameters as optimal for the binary random forest model:

- `n_estimators`: 500
- `max_depth`: 20
- `min_samples_leaf`: 1

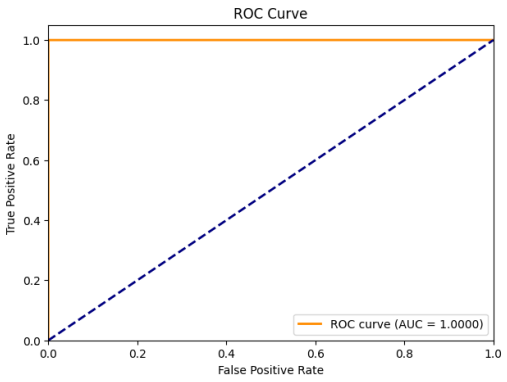
It is important to state that at the beginning of the project, a portion of the dataset was extracted using a train/test split to use during this final test to prevent any type of data leakage.

6.2 Model Evaluation and Metrics

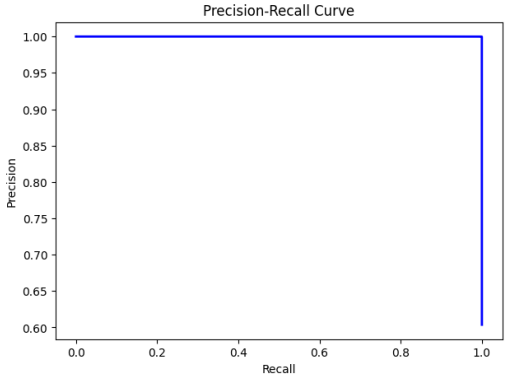
To provide an evaluation of the classifier, especially in the presence of strong class imbalance, some key metrics are reported.



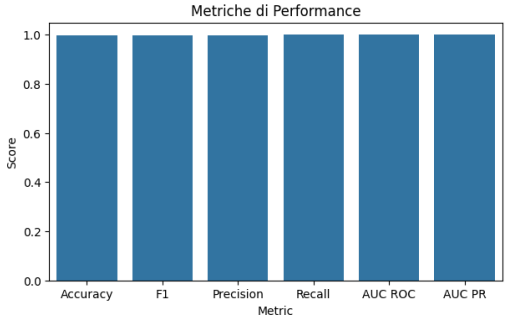
(a) Confusion matrix on the test set.



(b) ROC Curve



(c) PR Curve



(d) Performance metrics

Figure 6.1: Summary of key evaluation metrics and visualizations for the optimized model.

The metrics are almost perfect, but it is necessary to check if the few values that were misclassified are actually the classes with a few samples, this type of problem can only be solved with a multiclass evaluation.

Chapter 7

Multiclass Classification Results

7.1 Pipeline Structure

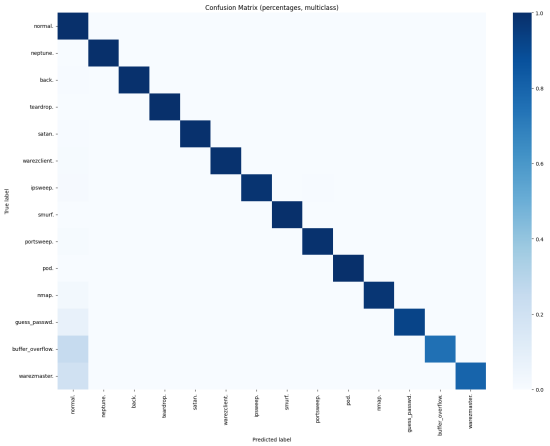
The pipeline is the same as the binary one, the key difference stands in the selected hyperparameters and obviously the target variable.

Best Parameter Choices: The grid search process identified the following hyperparameters as optimal for the multiclass random forest model:

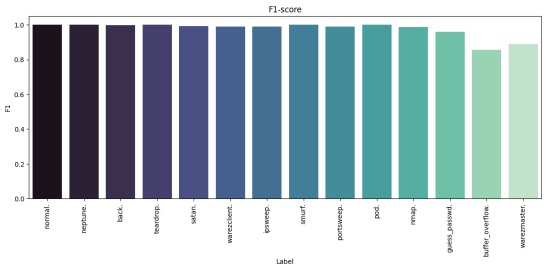
- `n_estimators`: 500
- `max_depth`: null
- `min_samples_leaf`: 5

The same test set used for the binary classification was used to evaluate the multiclass problem.

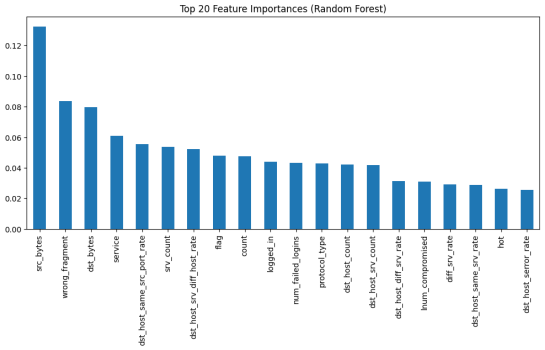
7.2 Multiclass Model Evaluation



(a) Normalized confusion matrix for the multiclass classification task.



(b) F1-score for each class, highlighting strong performance even for rare attacks.



(c) Top 20 feature importances for the multiclass task.

Figure 7.1: Summary of multiclass performance.

The multiclass confusion matrix (a) confirms strong diagonal dominance, indicating accurate classification of each attack type. The per-class F1-scores (b) highlight robust detection across nearly all classes, though the rarest attacks remain the most challenging, but it is important to remember that these type of attack have approximately *15 to 40 samples* in the training set. The macro-averaged F1-score for this experiment is **0.9754**. Feature importances (c) reveal which inputs drive the model's performance in the multiclass scenario and the main one is *src_bytes*, but it still stays at *0.12*.

7.3 Test with lower minimum samples

When testing with a lower minimum support threshold (10 samples), the model struggled to correctly classify classes with extremely low support. These minority classes did not have sufficient samples, leading to poor generalization performance.

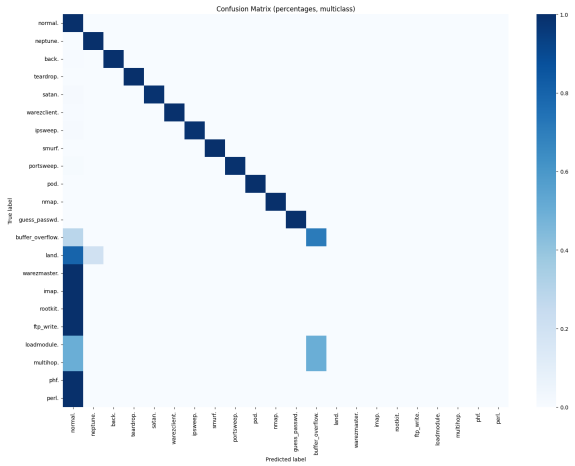


Figure 7.2: Test with min_samples=10

Chapter 8

Conclusions

8.1 GUI

It is possible to visualize the results of the pipelines in a simple GUI in the file *gui.py* and use the provided test set.

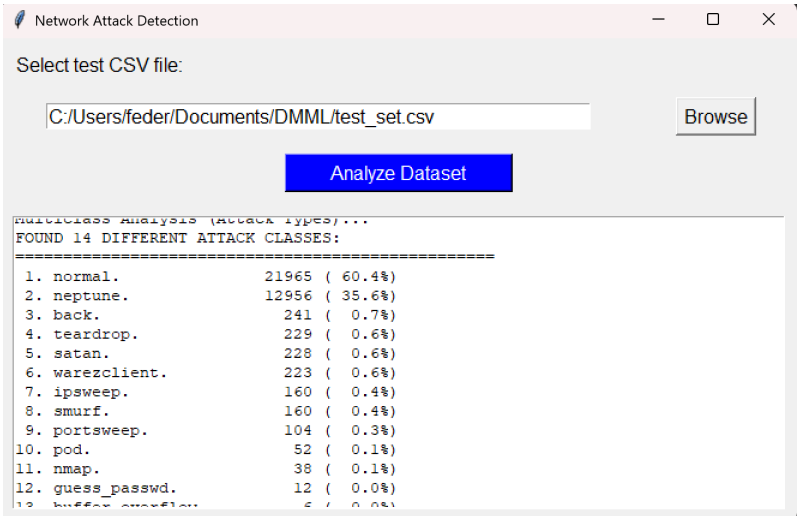


Figure 8.1: Simple GUI screenshot

8.2 Strengths and Weaknesses

The final pipeline demonstrated strong performance in both binary and multiclass intrusion detection, achieving high macro-averaged F1 scores and robust detection of both frequent and rare attack types. The structured, modular use of `scikit-learn` pipelines guaranteed reproducibility, eliminated data leakage, and enabled efficient hyperparameter optimization.

Strengths:

- **High Classification Performance:** The optimized random forest pipeline

consistently achieved superior F1 macro scores, reflecting balanced performance across all classes, not only the majority ones.

- **Data Leakage Prevention and Reproducibility:** All data transformations, feature selection, and class balancing were encapsulated within the pipeline and performed inside the cross-validation folds, ensuring reproducible results.

Weaknesses and Limitations:

- **Resource Constraints:** Hardware limitations (CPU, RAM) restricted the breadth of hyperparameter optimization. The grid search was necessarily reduced to a subset of the most promising parameters, possibly missing the true global optimum.
- **Long Training Times:** The combination of SMOTE, repeated feature selection, and cross-validation with a large dataset led to extended training durations, limiting the feasibility of exhaustive searches or frequent re-tuning on a limited hardware.
- **Rare Classes:** Despite the use of SMOTENC, some extremely rare attack types in KDD99 remained underrepresented or difficult to classify, due to the limited original sample size and potential for overfitting in minority oversampling.